

OBSERVACIONES DE LA PRÁCTICA

1. Nicolás Ramírez Correa, n.ramirez23@uniandes.edu.co, 202610819
2. Samuel Pinto Pedraza, s.pintop2@uniandes.edu.co, 202610615
3. Juan Diego Villa Garcia, jd.villa@uniandes.edu.co, 202615562

Máquina 1	
Procesador	AMD Ryzen 7 7700 8-Core Processor
Memoria RAM (GB)	32 GB
Sistema Operativo	Microsoft Windows 11 Pro - 64 bits

Tabla 1. Especificaciones de la máquina para ejecutar las pruebas de rendimiento.

Resultados

Carga de Catálogo LINEAR PROBING

Factor de Carga (PROBING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @LP [ms]
0.1	389519.74	3908.29
0.5	379504.16	3667.50
0.7	378789.82	3763.35
0.9	378396.00	4022.93

Tabla 2. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando LINEAR PROBING

Carga de Catálogo SEPARATE CHAINING

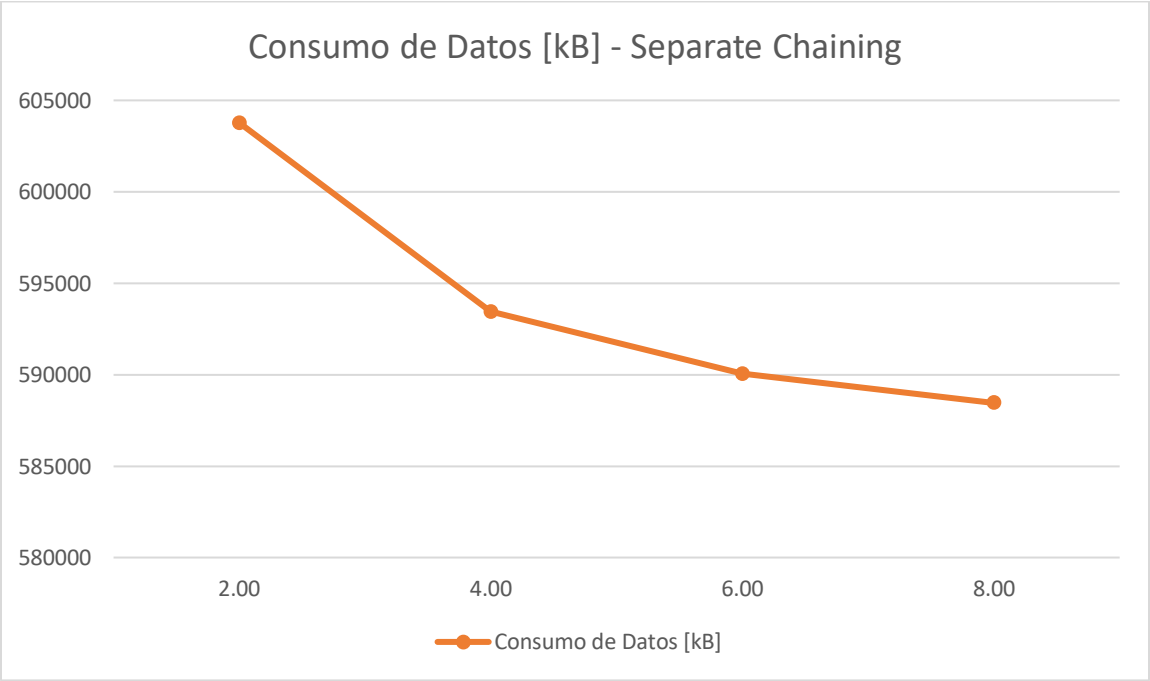
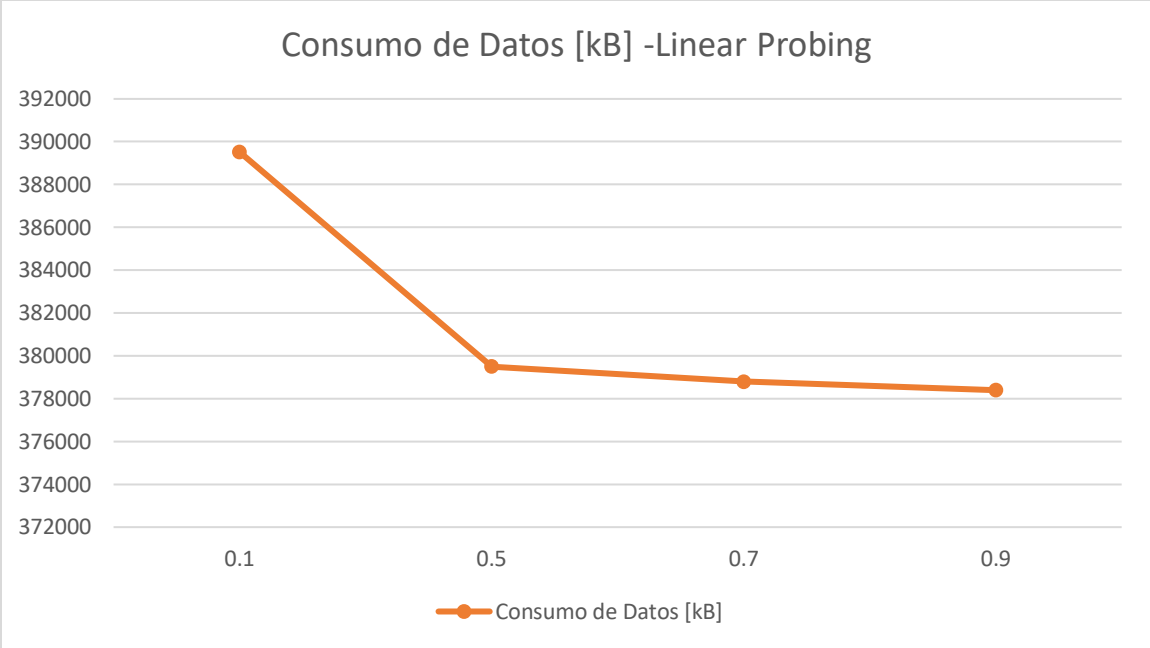
Factor de Carga (CHAINING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @SC [ms]
2.00	603773.09	4257.20
4.00	593439.19	4568.70
6.00	590043.61	4584.80
8.00	588451.89	4629.35

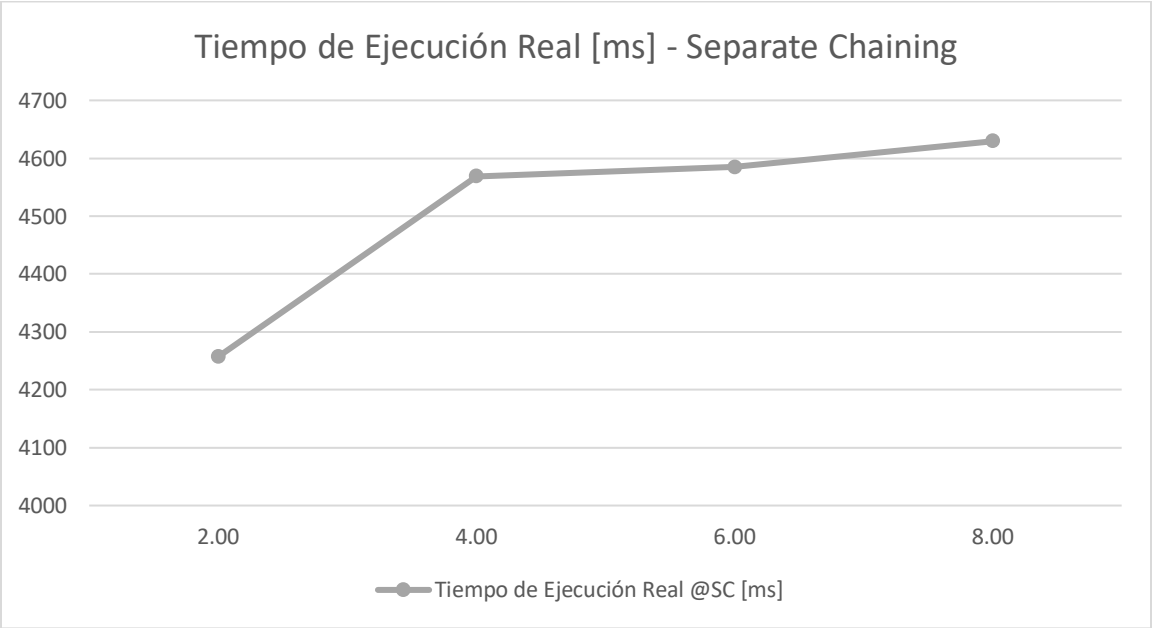
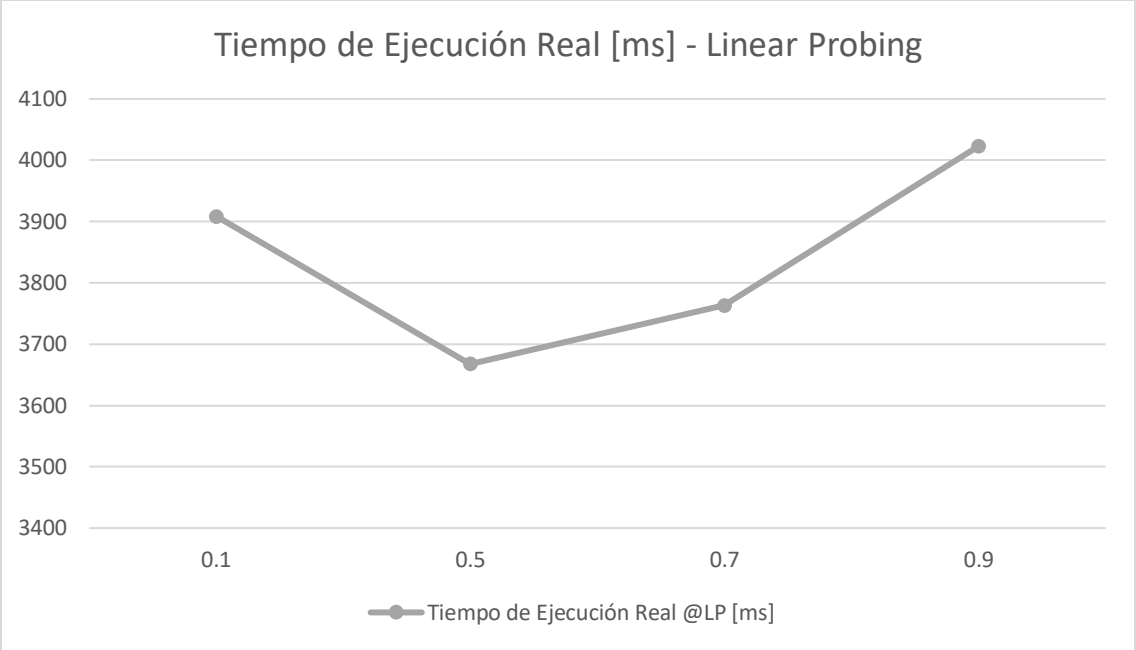
Tabla 3. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando SEPARATE CHAINING

Gráficas

La gráfica generada por los resultados de las pruebas de rendimiento.

- Comparación de memoria y tiempo de ejecución para LINEAR PROBING y SEPARATE CHAINING
- Comparación de factor de carga y memoria para LINEAR PROBING y SEPARATE CHAINING





Preguntas de análisis

1. ¿Por qué en la función **getTime()** se utiliza **time.perf_counter()** en vez de otras funciones como **time.process_time()**?

Se utiliza `time.perf_counter()` porque permite medir el tiempo real transcurrido durante la ejecución de una operación. En cambio, `time.process_time()` mide únicamente el tiempo que el procesador dedica al programa. Para estas pruebas interesa conocer cuánto tarda realmente la carga del catálogo desde que inicia hasta que termina.

2. ¿Por qué son importantes las funciones **start()** y **stop()** de la librería **tracemalloc**?

`tracemalloc.start()` permite comenzar a registrar las asignaciones de memoria realizadas por Python durante la ejecución de la prueba. Después, `tracemalloc.stop()` detiene ese seguimiento. Esto permite calcular la memoria utilizada durante una operación específica y evita mantener el monitoreo activo durante partes del programa que no hacen parte de la medición.

3. ¿Por qué no se puede medir paralelamente el **uso de memoria** y el **tiempo de ejecución** de las operaciones?

No es conveniente medirlos simultáneamente porque el monitoreo de memoria mediante `tracemalloc` agrega trabajo adicional al programa. Esto puede aumentar artificialmente el tiempo de ejecución. Por esta razón se hicieron pruebas separadas para tiempo y memoria, obteniendo mediciones más representativas.

4. Dado el número de elementos de los archivos (**large**), ¿Cuál sería el factor de carga para estos índices según su mecanismo de colisión?

Para Linear Probing, el factor de carga debe mantenerse menor que 1, ya que los elementos se almacenan directamente en las posiciones de la tabla y, a medida que esta se llena, aumentan las colisiones. En las pruebas realizadas se utilizaron factores de 0.1, 0.5, 0.7 y 0.9. Para Separate Chaining, el factor de carga puede ser mayor que 1, porque cada posición de la tabla puede almacenar varios elementos dentro de una misma cadena. Por esta razón se probaron factores de 2.0, 4.0, 6.0 y 8.0.

5. ¿Qué cambios percibe en el tiempo de ejecución al modificar el factor de carga máximo?

En Linear Probing no se observó una relación completamente lineal. El menor tiempo se obtuvo con factor 0.5, con 3667.50 ms, mientras que con factor 0.9 el tiempo aumentó hasta 4022.93 ms. Esto muestra que una tabla demasiado llena puede aumentar el costo de las operaciones por las colisiones. En Separate Chaining también se observó un aumento del tiempo al incrementar el factor de carga. El tiempo pasó de 4257.20 ms con factor 2.0 a 4629.35 ms con factor 8.0. Esto puede explicarse porque un mayor factor de carga produce cadenas más largas y, por tanto, más elementos que deben recorrerse dentro de un mismo bucket.

6. ¿Qué cambios percibe en el consumo de memoria al modificar el factor de carga máximo?

En ambos mecanismos, al aumentar el factor de carga disminuyó el consumo de memoria. En Linear Probing pasó de 389519.74 kB con factor 0.1 a 378396.00 kB con factor 0.9.

En Separate Chaining pasó de 603773.09 kB con factor 2.0 a 588451.89 kB con factor 8.0. Esto ocurre porque un factor de carga mayor permite almacenar más elementos respecto al número de posiciones disponibles en la estructura.

7. ¿Qué cambios percibe en el tiempo de ejecución al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

En nuestras pruebas, Linear Probing presentó menores tiempos de carga en general que Separate Chaining. Los tiempos de Linear Probing estuvieron entre 3667.50 ms y 4022.93 ms, mientras que Separate Chaining estuvo entre 4257.20 ms y 4629.35 ms.

Esto puede relacionarse con que Separate Chaining utiliza estructuras adicionales para almacenar los elementos dentro de cada bucket, mientras que Linear Probing trabaja directamente sobre posiciones de una tabla.

8. ¿Qué cambios percibe en el consumo de memoria al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

Separate Chaining utilizó considerablemente más memoria que Linear Probing en todas las pruebas. Linear Probing estuvo aproximadamente entre 378396 kB y 389520 kB, mientras que Separate Chaining estuvo entre 588452 kB y 603773 kB.

Esto se debe a que Separate Chaining necesita mantener una estructura adicional para cada bucket y almacenar listas con los elementos que presentan colisiones, mientras que Linear Probing almacena los elementos directamente dentro de la tabla.